

VPA A2 S2 Weeks 10 – 18: Revision & Final Exam Preparation

Victoria Park Academy — A2 Level Computer Science

Semester 2, Weeks 10 – 18: Intensive Revision & Final Exam Preparation

Duration: 9 Weeks (Weeks 10 – 18) | Final Exams: May/June 2026

Paper 1: 1h 45m, 75 marks (Theory) | Paper 2: 2h, 75 marks (Programming)

Strategy: Revise → Mock → Analyse → Target → Revise → Mock → Final Polish

Overview of the Revision Phase

Weeks 10–18 represent the final sprint to A-Level examinations. The focus shifts from project work to systematic, evidence-based revision. Every lesson is designed to close gaps, build confidence, and refine exam technique.

Revision Philosophy:

- ☐ Retrieval Practice: Pull information from memory, don't just re-read.
- ☐ Spaced Repetition: Review topics at increasing intervals.
- ☐ Interleaving: Mix topics to strengthen discrimination.
- ☐ Dual Coding: Combine words and visuals.
- ☐ Elaboration: Explain ideas in detail to deepen understanding.
- ☐ Concrete Examples: Anchor abstract concepts in real-world instances.

Phase 1: Paper 1 Revision (Weeks 10 – 12)

Focus: All theory topics from AS and A2, with emphasis on high-tariff questions and common misconceptions.

Week 10: Data Representation, Hardware & Software, Communication

Day 1: Data Representation

Binary, denary, hexadecimal conversions (quick-fire drills).

Two's complement: representation, range, overflow detection.

Floating-point representation: mantissa, exponent, normalisation.

Binary-coded decimal (BCD), Gray code, ASCII/Unicode.

Bitmap and vector graphics: colour depth, resolution, file size calculations.

Sound: sampling rate, sampling resolution, bit rate, Nyquist theorem.

Activity: 20-minute past paper question on floating-point normalisation. Peer mark using mark scheme.

Day 2: Hardware & Processor Fundamentals

Von Neumann architecture: fetch-decode-execute cycle, registers (PC, MAR, MDR, ACC, CIR).

Assembly language: instruction set, addressing modes, assembly to machine code.

Factors affecting processor performance: clock speed, word length, cache, cores, pipelining.

Input, output, and storage devices: characteristics, suitability, emerging technologies.

Activity: Trace a fetch-decode-execute cycle on a given assembly program. Calculate execution time with and without pipelining.

Day 3: Software & System Software

Operating systems: memory management (paging, segmentation, virtual memory), scheduling algorithms.

Translation software: compilers, interpreters, assemblers. Lexical analysis, syntax analysis, code generation, optimisation.

Utility programs: backup, defragmentation, compression, virus checkers.

Activity: Compare paging and segmentation in a structured table. Evaluate which is better for a given scenario.

Day 4: Communication & Internet Technologies

Network types, topologies, protocols (TCP/IP, HTTP, FTP, SMTP).

Client-server vs. peer-to-peer. Thin vs. thick client.

Circuit switching, packet switching, routing algorithms.

Network security: firewalls, proxies, encryption, certificates.

Activity: Explain how packet switching improves reliability compared to circuit switching. Use a diagram.

Day 5: Consolidation & Mock Question Practice

Mixed topic past paper questions (2019 – 2024).

Peer assessment using mark schemes.

Teacher-led clinic on weakest areas identified.

I can convert between binary, denary, and hexadecimal in under 30 seconds

I can normalise a floating-point number

I can trace the fetch-decode-execute cycle

I can explain paging, segmentation, and virtual memory

I can describe the stages of compilation

I can compare network topologies and protocols

Week 11: Security, AI, Ethics & Advanced Theory

Day 1: Security & Privacy

Malware: viruses, worms, trojans, spyware, ransomware. Prevention and detection.

Encryption: symmetric, asymmetric, digital signatures, digital certificates, PKI.

Authentication: passwords, biometrics, two-factor authentication, CAPTCHA.

Data protection: GDPR principles, data subjects, controllers, processors.

Activity: Evaluate the use of biometric authentication vs. password authentication for a banking app.

Day 2: Artificial Intelligence

Machine learning: supervised, unsupervised, reinforcement learning.

Neural networks: perceptron, multi-layer, backpropagation, deep learning.

Training data: feature extraction, training set, validation set, test set, overfitting, underfitting.

Applications: natural language processing, computer vision, expert systems, robotics.

Ethical implications: bias, transparency, accountability, job displacement.

Activity: Explain how a neural network learns using backpropagation. Draw a diagram.

Day 3: Ethics & Legal Issues

Computer misuse: Computer Misuse Act, hacking, unauthorised access.

Intellectual property: copyright, patents, trademarks, software licensing (GPL, proprietary).

Ethical frameworks: consequentialism, deontology, virtue ethics applied to computing.

Environmental impact: e-waste, energy consumption, green computing.

Activity: Discuss the ethical implications of AI in criminal sentencing. Use a structured argument.

Day 4: Advanced Theory (A2 Topics)

Boolean algebra: simplification, Karnaugh maps, De Morgan's laws, logic gates.

Flip-flops, registers, counters. Sequential logic vs. combinational logic.

Big data: characteristics (volume, velocity, variety, veracity), technologies (Hadoop, Spark), applications.

Robotics: sensors, actuators, feedback loops, autonomous systems.

Activity: Simplify a Boolean expression using a Karnaugh map. Design a logic circuit.

Day 5: Consolidation & Extended Answer Practice

Focus on 8–12 mark "Evaluate" and "Discuss" questions.

Peer assessment with structured feedback.

Teacher modelling of top-band responses.

I can explain symmetric and asymmetric encryption with examples

I can describe how a neural network learns

I can discuss ethical issues in AI with balanced arguments

I can simplify Boolean expressions using Karnaugh maps

I can evaluate the environmental impact of computing

I can write a structured 12-mark evaluation

Week 12: Paper 1 Mock Exam & Post-Mortem

Day 1: Mock Exam 2 — Paper 1 (1h 45m, 75 marks)

Full exam conditions: silent room, no notes, timed strictly.

Question paper: past paper (2023 or 2024) or teacher-generated equivalent.

Students must bring: pen, pencil, ruler, calculator (non-programmable).

Day 2: Self-Assessment & Marking

Students mark their own paper using the official mark scheme.

Complete a self-reflection sheet: what went well, what needs improvement.

Identify top 3 areas for targeted revision.

Day 3: Teacher-Led Post-Mortem

Class analysis of common errors and high-scoring approaches.

Focus on questions where the class average was below 50%.

Modelling of correct answers for high-tariff questions.

Day 4: Targeted Intervention

Small-group clinics based on mock performance.

Individual target sheets: specific actions for each student.

Additional practice questions for weak areas.

Day 5: Consolidation & Confidence Building

Retrieval practice quiz on Paper 1 topics.

Peer teaching: students explain a topic they now understand to a peer who is still struggling.

Positive reinforcement: celebrate improvements since Mock 1.

I have sat Mock 2 under full exam conditions

I have marked my paper and identified my score

I know my top 3 weak areas

I have a targeted action plan

I have attended a clinic for at least one weak area

Phase 2: Paper 2 Revision (Weeks 13 – 14)

Focus: Programming concepts, problem solving, algorithms, and data structures. Paper 2 is 2 hours of sustained problem-solving.

Week 13: Programming Fundamentals & Problem Solving

Day 1: Programming Constructs & Data Types

Variables, constants, data types (integer, real, Boolean, character, string, array, record).

Assignment, arithmetic operators, relational operators, logical operators.

Selection: IF, CASE, nested selection.

Iteration: FOR, WHILE, REPEAT, nested loops.

Built-in functions: string manipulation, mathematical functions, random numbers.

Activity: Write pseudocode for a program that validates a password according to specific rules. Trace the logic.

Day 2: Structured Programming & Subroutines

Procedures and functions: parameters, return values, local vs. global variables.

Modular design: top-down design, stepwise refinement.

Recursion: base case, recursive case, stack frames, tail recursion.

Parameter passing: by value, by reference.

Activity: Write a recursive function to calculate factorial. Trace the call stack for factorial(5).

Day 3: File Handling & Data Structures

File operations: open, read, write, append, close. CSV and text file handling.

1D arrays: declaration, initialisation, traversal, searching, sorting.

2D arrays: declaration, traversal, applications (e.g., game boards, matrices).

Records: composite data types, arrays of records.

Linked lists, stacks, queues, trees: operations, implementations, applications.

Activity: Write a program to read a CSV file, store data in an array of records, and perform a binary search.

Day 4: Object-Oriented Programming

Classes, objects, attributes, methods.

Encapsulation: private/public attributes, getters, setters.

Inheritance: base class, derived class, overriding, polymorphism.

Class diagrams: UML notation.

Activity: Design a class hierarchy for a library system. Write the Python code for the base class and one derived class.

Day 5: Consolidation & Past Paper Practice

Mixed programming questions from past papers (2019 – 2024).

Timed practice: 20-minute questions under exam conditions.

Peer code review: mark each other's pseudocode against the mark scheme.

I can write pseudocode for any standard programming construct

I can explain recursion and trace a recursive call

I can implement file handling in Python

I can use arrays, records, and linked lists appropriately

I can design and implement OOP classes

I can complete a 20-mark programming question in 25 minutes

Week 14: Algorithms, Computational Thinking & Paper 2 Mock

Day 1: Searching & Sorting Algorithms

Linear search: algorithm, complexity ($O(n)$), best/worst/average case.

Binary search: algorithm, complexity ($O(\log n)$), prerequisite (sorted data).

Bubble sort: algorithm, complexity ($O(n^2)$), optimisation (flag).

Insertion sort: algorithm, complexity ($O(n^2)$), best case ($O(n)$).

Quicksort: algorithm, complexity ($O(n \log n)$ average, $O(n^2)$ worst), pivot selection.

Merge sort: algorithm, complexity ($O(n \log n)$), divide and conquer.

Activity: Trace bubble sort and quicksort on a given array. Compare the number of comparisons and swaps.

Day 2: Computational Thinking & Algorithm Design

Decomposition: breaking complex problems into manageable parts.

Pattern recognition: identifying similarities and trends.

Abstraction: hiding unnecessary detail, focusing on relevant features.

Algorithm design: pseudocode, flowcharts, structure diagrams.

Validation and verification: range checks, type checks, presence checks, consistency checks.

Activity: Given a complex scenario (e.g., a booking system), decompose it into modules and design algorithms for each.

Day 3: Mock Exam 3 — Paper 2 (2h, 75 marks)

Full exam conditions: silent room, no notes, timed strictly.

Students must bring: pen, pencil, ruler, calculator.

Question paper: past paper (2023 or 2024) or teacher-generated equivalent.

Day 4: Post-Mortem & Targeted Intervention

Self-assessment using mark scheme.

Class analysis of programming errors (logic errors, syntax errors, off-by-one errors).

Teacher-led modelling of correct algorithmic solutions.

Individual target sheets with specific programming practice tasks.

Day 5: Consolidation & Confidence Building

Retrieval practice: flashcard drill on all programming constructs.

Peer teaching: explain an algorithm to a partner who found it difficult.

Celebrate progress: highlight students who have improved significantly.

I can trace any searching or sorting algorithm

I can compare algorithm complexity using Big-O notation

I have sat Mock 3 under full exam conditions

I have identified my programming weaknesses

I have a practice plan for my weak areas

Phase 3: Mock Series 2 (Weeks 15 – 16)

Focus: Full A-Level mock exams under strict exam conditions. These are the final diagnostic before the real exams.

Week 15: Mock Exam 4 — Paper 1 (1h 45m, 75 marks)

Day 1: Mock Exam 4 — Paper 1

Full exam conditions. No notes. No communication.

Latest available past paper or secure teacher-generated paper.

Strict timing: 1h 45m. No extra time unless officially agreed.

Day 2: Marking & Feedback

Teacher marks papers using official mark scheme.

Individual feedback sheets: mark per question, comments, target grade comparison.

Class data: average per question, areas of strength, areas for concern.

Day 3: Targeted Revision

Students work on their weakest topics using personalised revision packs.

Teacher runs drop-in clinics for students scoring below target.

Peer support: students who scored highly mentor those who struggled.

Day 4: Extended Answer Workshop

Focus on 8–12 mark questions: structure, evidence, judgement.

Students write a model answer in pairs. Teacher selects best examples to share.

Practice writing introductions and conclusions that directly address the question.

Day 5: Consolidation

Quick-fire quiz on all Paper 1 topics.

Revision of key definitions using bilingual glossary.

Positive mindset activity: visualising success in the exam.

Week 16: Mock Exam 5 — Paper 2 (2h, 75 marks)

Day 1: Mock Exam 5 — Paper 2

Full exam conditions. No notes. No communication.

Latest available past paper or secure teacher-generated paper.

Strict timing: 2h. No extra time unless officially agreed.

Day 2: Marking & Feedback

Teacher marks papers using official mark scheme.

Individual feedback sheets: mark per question, comments, target grade comparison.

Class data: programming errors, algorithm design weaknesses, time management issues.

Day 3: Programming Clinic

Intensive programming practice on topics where the class underperformed.

Timed 20-minute questions to build speed and accuracy.

Debugging exercises: find and fix errors in given code.

Day 4: Final Gap Analysis

Each student completes a final gap analysis: what they know vs. what they need to know.

Teacher reviews each gap analysis and approves final revision priorities.

Personalised revision timetable created for the final two weeks.

Day 5: Celebration & Reset

Celebrate effort and progress. Acknowledge how far students have come.

Reset mindset: mocks are practice, not prophecy. Focus on what can still be improved.

Introduce the final two-week plan: what to revise, when, and how.

Phase 4: Final Preparation (Weeks 17 – 18)

Focus: Confidence building, exam logistics, wellbeing, and final targeted revision. No new content.

Week 17: Final Revision & Confidence Building

Day 1: Paper 1 Final Polish

Quick revision of all Paper 1 topics using condensed notes (one page per topic).

Focus on high-frequency topics: data representation, hardware, security, AI.

Practice 1-mark and 2-mark questions for speed.

Day 2: Paper 2 Final Polish

Quick revision of all programming constructs and algorithms.

Practice tracing code: given a program, predict the output.

Practice writing pseudocode from a description.

Day 3: Exam Technique Masterclass

Time management: how to allocate time in each paper.

Command words: final drill on what each word requires.

Structure for extended answers: PEEL (Point, Evidence, Explanation, Link).

What to do if you get stuck: skip, flag, return.

Checking strategy: 5 minutes at the end for silly errors.

Day 4: Wellbeing & Stress Management

Sleep hygiene: importance of 8+ hours before exams.

Nutrition: brain food (omega-3, hydration, balanced meals).

Exercise: light activity to reduce cortisol and improve focus.

Mindfulness: breathing exercises, visualisation, positive self-talk.

Balanced revision: 45-minute blocks with 10-minute breaks.

Day 5: Q&A; and Open Revision

Student-led Q&A;: any topic, any question.

Open revision: students work on their personalised revision timetables.

Teacher circulates for one-to-one support.

I have condensed notes for every topic (one page each)

I know how to manage my time in each paper
I can identify every command word and what it requires
I have a sleep and nutrition plan for exam week
I have a personalised revision timetable for the final week

Week 18: Exam Week Logistics & Final Preparation

Day 1: Paper 1 Exam Day Preparation

What to bring: pens (black ink), pencils, eraser, ruler, calculator, watch, water (clear bottle, no label).

What NOT to bring: mobile phone, smartwatch, notes, food.

Arrival time: 30 minutes before the exam starts.

Exam room protocol: silence, raise hand for questions, no leaving in first or last 15 minutes.

Reading time: use the first 5 minutes to scan the paper and plan your attack.

Day 2: Paper 2 Exam Day Preparation

Same logistics as Paper 1.

Additional focus: programming questions require clear pseudocode or Python.

Diagrams: use a pencil for flowcharts and structure diagrams.

Extra paper: ask for more if needed; clearly label all additional sheets.

Day 3: Final Retrieval Practice

Flashcard session: 100 most important terms.

Quick-fire quiz: 50 questions in 30 minutes.

Peer challenge: one student asks, another answers.

Day 4: Positive Visualisation & Relaxation

Guided visualisation: walking through the exam day from waking up to handing in the paper.

Positive affirmations: "I am prepared. I am capable. I will succeed."

Relaxation techniques: progressive muscle relaxation, box breathing.

Day 5: Final Briefing & Farewell

Final reminders: logistics, equipment, what to expect.

Teacher message: "You have worked incredibly hard. Trust your preparation."

Students leave with a revision pack for the final days before each exam.

Contact details: how to reach the teacher if last-minute questions arise.

☐ You are ready. Trust the process. Trust yourself. Go and succeed.

Resources

Cambridge A-Level Computer Science (9618) Past Papers (2018 – 2025)

Cambridge Examiner Reports (2018 – 2025)

Condensed Revision Notes (one page per topic) — provided by teacher

Bilingual Glossary (150 Terms) — Supporting Documents

Flashcard App (Anki or Quizlet) — student-managed

Python 3.12 — for programming practice

VS Code — for code writing and testing

Document Control: Version 1.0 | Created: January 2026 | Owner: VPA Computer Science Department

Next Review: June 2026 (post-exam analysis)